

RISC-V Multi-Cycle CPU: Extended Instruction Set

December 3, 2025

Previously, we took a provided single-cycle RISC-V CPU written in SystemVerilog, and transformed it into a RISC-V multi-cycle CPU. Both of these iterations had an instruction set of 13 total instructions: AND, ANDI, OR, ORI, ADD, ADDI, SUB, SLT, SLTI, BEQ, LW, SW, and JAL. Our final project focuses on expanding this instruction set with most of the other standard instructions in RISC-V. The goal was to double the instruction count with 13 additional instructions, with a focus on functionally significant instructions that are a bit problematic to not have available. Additionally, we will verify its successful implementation onto an Altera DE2-115 FPGA by displaying information on the LEDs and seven-segment displays.

The extended instruction set now includes several new R-type instructions, and each of these also has their immediate versions implemented as well. We added XOR, along with the shifting operations: SLL, SRL, SRA. These were mostly simple in their implementation. Our ALU already had all of these operations except SRA, so the instruction decoding simply had to route them there. To accommodate SRA, we just had to add another bit to ALUControl, making it 4 bits. No new states required!

Also new are the three new branch instructions: BNE, BGE, and BLT, or: branch if not equal, branch if greater than or equal, and branch if less than. The necessary information for these new conditions was actually just a single one-bit signal from the ALU to the Control we called “lessThan,” in combination with the already existing “Zero” signal. You can see it in the system diagram (Fig. 1 below). In the datapath (Fig. 2), it simply runs alongside the existing zero signal.

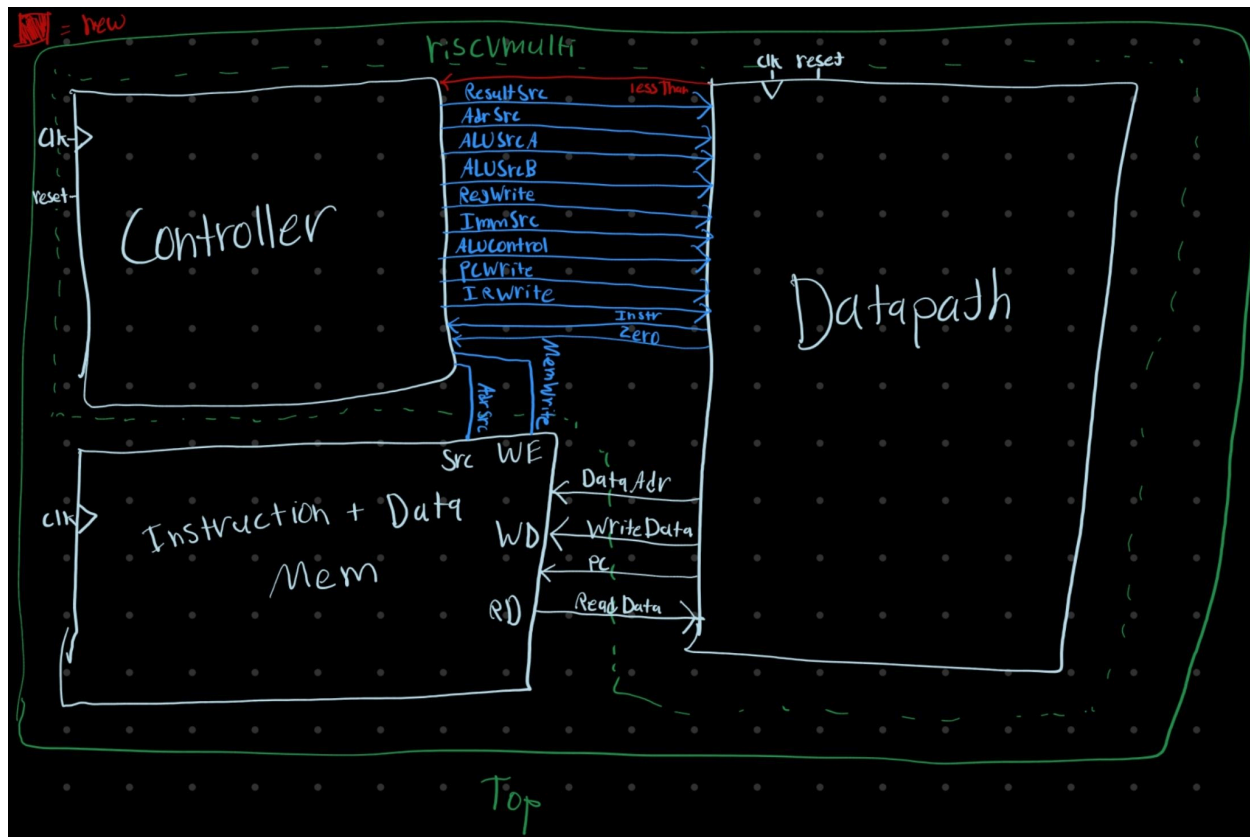


Fig. 1. Multi-Cycle CPU System Diagram

Given that new bit of information, we used one-hot encoding in the control unit to store which branch is currently being attempted. BEQ, BNE, BLT, and BGE correspond to 0001, 0010, 0100, and 1000 when encoded. The slight change to the “BEQ” state, now called the “Branch” state, is reflected in the controller finite state machine diagram in Fig. 3 below.

The second tricky instruction is JALR (jump and link to register + immediate). JALR required the existing functionality of storing PC + 4 into the destination register. Instead of jumping to a target address, it should jump to a source register plus a sign extended immediately. So, we added state 12: JALR, which performs the register and immediate summing. Then, it follows into the standard jump and link logic to finish its task.

With that, our 13 instructions were implemented, and it was time to test them out! We wrote an almost overly extensive [test program](#) that utilizes every instruction in the set, with the exception of ones that have both I-type and R-type versions (for those we just used one or the other).

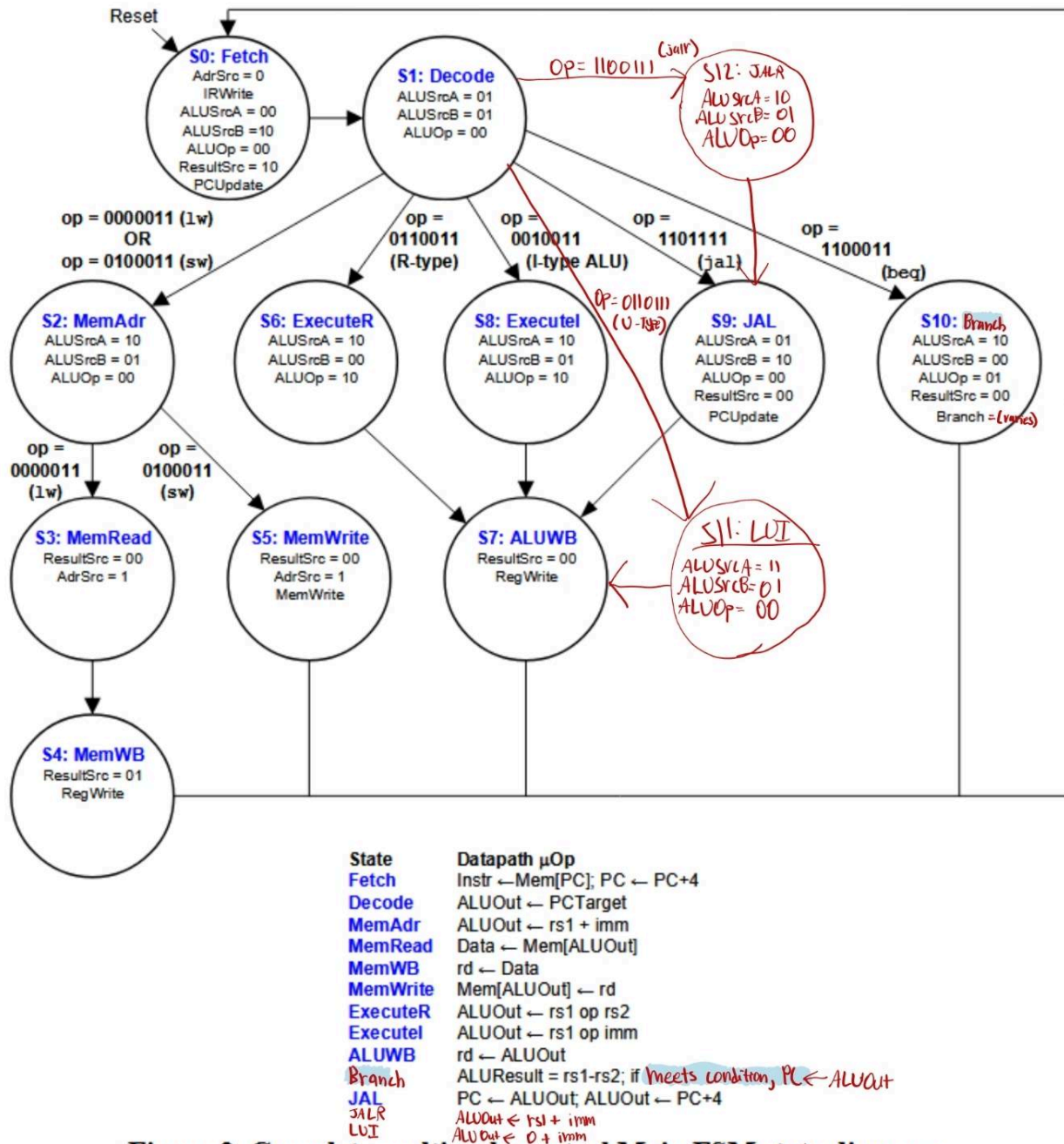


Fig. 3. Multi-Cycle CPU Controller State Transition Diagram

The results of our test program are shown through the simulation waveforms in Fig. 4. After some bug fixing, everything seems to be working as expected. It is tiring to trace through the simulation waveform checking every result, (though we did) so here are some key points. At PC = 0x4, we should see a result of 0x12345000 as a successful LUI operation, and should see it turn into 0x12345678 at PC = 0x8. Then, after various simple operations, we will jump and link to a function that simply performs a shift right logical. We quickly jump from PC = 0x30 to PC = 0x5c, and get the expected shift result of 0x00002468 at PC = 0x60. We soon return back to PC = 0x30 to continue operation. After some more branches and operations, we will branch to the

end of the program and loop there forever. This happens at the end of the simulation, when we reach PC = 0x68, but the looping is not visible since we ended the [testbench](#) at that spot.

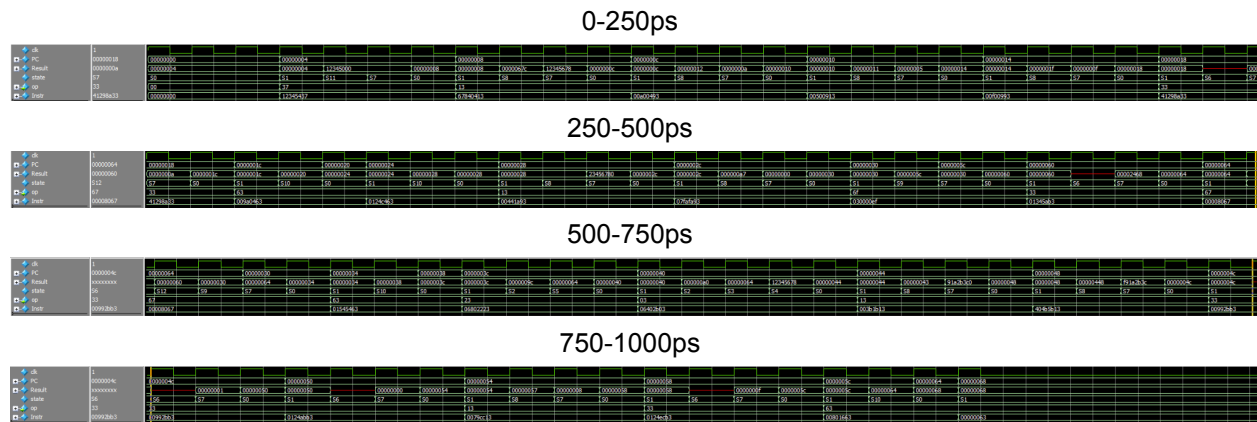


Fig. 4. Test Program Simulation Waveform

Finally, we created a [wrapper](#) to display the result of each operation on the seven-segment displays, and the current PC in binary on the green LEDs. This will be programmed to the DE2 board to allow easy stepping through on a real FPGA. We assigned a push button to the clock, and a switch to the reset signals, and it performed as expected in our live demonstration!

Through implementing 13 additional instructions to a limited instruction set, we gained significant respect for the architecture's simplicity, since adding to even just a minimal implementation of RISC-V was simple and didn't require any more modules. Through our lengthy test program we were reminded that simulation is a great way to figure out what's going wrong with hardware description language code. We are very happy with the results, and could imagine going even further with [this implementation](#), adding instructions like auipc, unsigned versions, etc.

Code

riscvextended.sv

```
// riscvextended.sv

// 11/22/2025
// Comments that say "do x now" state an important difference to
riscvsingle.sv by Sarah Harris and David Harris

// 32-bit registers with instruction set:
// lw, sw
// add, sub, and, or, slt
// addi, andi, ori, slti
// beq, jal

// Extended adds the following instructions:
// sll slli srl srli
// xor xori
// bne blt bge jalr
// sra srai
// lui

module top(input logic clk, reset,
           output logic [31:0] WriteData, DataAdr, PC, Result,
           output logic MemWrite);

    logic AddrSrc;
    logic [31:0] ReadData;
    // instantiate cpu, mem unit
    riscvmulti rvmulti(clk, reset, PC, MemWrite, WriteData, ReadData,
DataAdr, Result, AddrSrc);
    idmem idmem(clk, MemWrite, AddrSrc, PC, DataAdr, WriteData, ReadData);

endmodule

module riscvmulti(input logic clk, reset,
                 output logic [31:0] PC,
                 output logic MemWrite,
                 output logic [31:0] WriteData,
                 input logic [31:0] ReadData,
                 output logic [31:0] DataAdr, Result, // DataAdr = Addr
                 output logic AddrSrc);
```

```

    logic      RegWrite, Jump, Zero, lessThan, PCWrite, IRWrite;
    logic [2:0] ImmSrc;
    logic [1:0] ResultSrc, ALUSrcA, ALUSrcB;
    logic [3:0] ALUControl;
    logic [31:0] Instr; // this comes from, dp now, and sends to controller

    controller c(clk, reset, Instr[6:0], Instr[14:12], Instr[30],
        Zero, lessThan, ImmSrc, ALUSrcA, ALUSrcB, ResultSrc,
        AddrSrc, ALUControl, IRWrite, PCWrite, RegWrite,
        MemWrite);
    datapath dp(clk, reset, ResultSrc, AddrSrc, ALUSrcA,
        ALUSrcB, RegWrite, ImmSrc, ALUControl, Zero, lessThan,
        PC, Instr, WriteData, ReadData, PCWrite,
        DataAdr, Result, IRWrite);

endmodule

module datapath(input logic      clk, reset,
    input logic [1:0] ResultSrc,
    input logic      AddrSrc,
    input logic [1:0] ALUSrcA, ALUSrcB,
    input logic      RegWrite,
    input logic [2:0] ImmSrc,
    input logic [3:0] ALUControl,
    output logic      Zero, lessThan,
    output logic [31:0] PC,
    output logic [31:0] Instr, // datapath outputs instr, very
diff to single cycle
    output logic [31:0] WriteData,
    input logic [31:0] ReadData,
    input logic      PCWrite,
    output logic [31:0] DataAdr, Result, // DataAdr = Addr,
Result = PCNext
    input logic      IRWrite);

    logic [31:0] OldPC, Data, ImmExt, rdA, rdB;
    logic [31:0] A, SrcA, SrcB, ALUResult, ALUOut;

    /* Instr/Data Memory Logic */

    eflopr #(32) pcreg(clk, reset, PCWrite, Result, PC); // pcnext -> pc reg
    mux2   #(32) pcmux(PC, Result, AddrSrc, DataAdr); // Addr mux
    flopr  #(32) datareg(clk, reset, ReadData, Data); // readdata -> data

```

```

reg
  eflopr #(32) oldpcreg(clk, reset, IRWrite, PC, OldPC); // pc -> oldpc
reg
  eflopr #(32) instrreg(clk, reset, IRWrite, ReadData, Instr); // readdata
-> instr reg

/* Register File Logic */

flopr #(32) areg(clk, reset, rdA, A); // rdA -> A reg
flopr #(32) breg(clk, reset, rdB, WriteData); // rdB -> Writedata reg
extend      ext(Instr[31:7], ImmSrc, ImmExt);
regfile      rf(clk, RegWrite, Instr[19:15], Instr[24:20], Instr[11:7],
Result, rdA, rdB);

/* ALU Logic */

alu          alu(SrcA, SrcB, ALUControl, ALUResult, Zero, lessThan);
mux3  #(32)  srcamux(PC, OldPC, A, ALUSrcA, SrcA); // SrcA mux
mux3  #(32)  srcbmux(WriteData, ImmExt, 32'd4, ALUSrcB, SrcB); // SrcB
mux
  flopr #(32) aluoutreg(clk, reset, ALUResult, ALUOut); // ALUResult ->
ALUOut reg
  mux3  #(32) resultmux(ALUOut, Data, ALUResult, ResultSrc, Result); //
Result mux

endmodule

/* Controller Module Definitions */

module controller(input logic clk, input logic reset,
                 input logic [6:0] op,
                 input logic [2:0] funct3,
                 input logic funct7b5,
                 input logic zero, lessThan,
                 output logic [2:0] immsrc,
                 output logic [1:0] ALUSrcA, ALUSrcB,
                 output logic [1:0] ResultSrc,
                 output logic AdrSrc,
                 output logic [3:0] alucontrol,
                 output logic IRWrite, PCWrite,
                 output logic RegWrite, MemWrite);

```

```

    logic PCUpdate;
    logic [3:0] Branch;
    logic [1:0] ALUOp;
    typedef enum bit[3:0] {S0,S1,S2,S3,S4,S5,S6,S7,S8,S9,S10,S11,S12}
State;
    State state;
    State nextState;

    aludec alu(op[5], funct3, funct7b5, ALUOp, alucontrol);
    instrdec instr(op, immsrc);

    always_ff @(posedge clk, posedge reset)
        if (reset) state = S0;
        else state = nextState;

    always_comb
        case(state)
            S0:    begin
                    AdrSrc = 0;
                    IRWrite = 1;
                    ALUSrcA = 2'b00;
                    ALUSrcB = 2'b10;
                    ALUOp = 2'b00;
                    ResultSrc = 2'b10;
                    PCUpdate = 1;
                    RegWrite = 0;
                    MemWrite = 0;
                    Branch = 4'b0000;
                    nextState = S1;
                end
            S1:    begin
                    ALUSrcA = 2'b01;
                    ALUSrcB = 2'b01;
                    ALUOp = 2'b00;
                    AdrSrc = 0;
                    IRWrite = 0;
                    ResultSrc = 0;
                    PCUpdate = 0;
                    RegWrite = 0;
                    MemWrite = 0;
                    Branch = 4'b0000;
                    case(op)

```

```

        7'b000_0011: nextState = S2; //I-Type
        7'b010_0011: nextState = S2; //S-Type
        7'b011_0011: nextState = S6; //R-Type
        7'b001_0011: nextState = S8; //I-Type
        7'b110_1111: nextState = S9; //jal
        7'b110_0111: nextState = S12; //jalr
        7'b110_0011: nextState = S10; //B-Type
        7'b011_0111: nextState = S11; //U-Type
        default: nextState = S0;
    endcase
end
S2:    begin
    ALUSrcA = 2'b10;
    ALUSrcB = 2'b01;
    ALUOp = 2'b00;
    AdrSrc = 0;
    IRWrite = 0;
    ResultSrc = 2'b00;
    PCUpdate = 0;
    RegWrite = 0;
    MemWrite = 0;
    Branch = 4'b0000;
    nextState = (op[5])? S5 : S3;
end
S3:    begin
    ResultSrc = 2'b00;
    AdrSrc = 1;
    ALUSrcA = 2'b00;
    ALUSrcB = 2'b00;
    ALUOp = 2'b00;
    IRWrite = 0;
    PCUpdate = 0;
    RegWrite = 0;
    MemWrite = 0;
    Branch = 4'b0000;
    nextState = S4;
end
S4:    begin
    ResultSrc = 2'b01;
    RegWrite = 1;
    AdrSrc = 0;
    ALUSrcA = 2'b00;
    ALUSrcB = 2'b00;

```

```

        ALUOp = 2'b0;
        IRWrite = 0;
        PCUpdate = 0;
        MemWrite = 0;
        Branch = 4'b0000;
        nextState = S0;
    end
S5:    begin
        ResultSrc = 2'b00;
        AdrSrc = 1;
        MemWrite = 1;
        ALUSrcA = 2'b00;
        ALUSrcB = 2'b00;
        ALUOp = 2'b00;
        IRWrite = 0;
        PCUpdate = 0;
        RegWrite = 0;
        Branch = 4'b0000;
        nextState = S0;
    end
S6:    begin
        ALUSrcA = 2'b10;
        ALUSrcB = 2'b00;
        ALUOp = 2'b10;
        AdrSrc = 0;
        IRWrite = 0;
        ResultSrc = 0;
        PCUpdate = 0;
        RegWrite = 0;
        MemWrite = 0;
        Branch = 4'b0000;
        nextState = S7;
    end
S7:    begin
        ResultSrc = 2'b00;
        RegWrite = 1;
        ALUSrcA = 2'b00;
        ALUSrcB = 2'b00;
        ALUOp = 2'b00;
        IRWrite = 0;
        PCUpdate = 0;
        MemWrite = 0;
        Branch = 4'b0000;

```

```

        AddrSrc = 0;
        nextState = S0;
    end
S8:    begin
        ALUSrcA = 2'b10;
        ALUSrcB = 2'b01;
        ALUOp = 2'b10;
        AddrSrc = 0;
        IRWrite = 0;
        ResultSrc = 2'b00;
        PCUpdate = 0;
        RegWrite = 0;
        MemWrite = 0;
        Branch = 4'b0000;
        nextState = S7;
    end
S9:    begin
        ALUSrcA = 2'b01;
        ALUSrcB = 2'b10;
        ALUOp = 2'b00;
        ResultSrc = 2'b00;
        PCUpdate = 1;
        AddrSrc = 0;
        IRWrite = 0;
        RegWrite = 0;
        MemWrite = 0;
        Branch = 4'b0000;
        nextState = S7;
    end
S10:begin
    case(func3)
        3'b000: Branch = 4'b0001; // beq
        3'b001: Branch = 4'b0010; // bne
        3'b100: Branch = 4'b0100; // blt
        3'b101: Branch = 4'b1000; // bge
    endcase
    ALUSrcA = 2'b10;
    ALUSrcB = 2'b00;
    ALUOp = 2'b01;
    ResultSrc = 2'b00;
    AddrSrc = 0;
    IRWrite = 0;
    PCUpdate = 0;

```

```

        RegWrite = 0;
        MemWrite = 0;
        nextState = S0;
    end
S11:begin
    ALUSrcA = 2'b11;
    ALUSrcB = 2'b01;
    ALUOp = 2'b00;
    AdrSrc = 0;
    IRWrite = 0;
    ResultSrc = 2'b00;
    PCUpdate = 0;
    RegWrite = 0;
    MemWrite = 0;
    Branch = 4'b0000;
    nextState = S7;
end
S12:begin
    ALUSrcA = 2'b10;
    ALUSrcB = 2'b01;
    ALUOp = 2'b00;
    AdrSrc = 0;
    IRWrite = 0;
    ResultSrc = 0;
    PCUpdate = 0;
    RegWrite = 0;
    MemWrite = 0;
    Branch = 4'b0000;
    nextState = S9;
end
endcase
    assign PCWrite = (Branch[0] & zero) | (Branch[1] & ~zero) | (Branch[2]
& lessThan) | (Branch[3] & ~ lessThan) | PCUpdate;
endmodule

module aludec(input logic opb5,
    input logic [2:0] funct3,
    input logic funct7b5,
    input logic [1:0] ALUOp,
    output logic [3:0] ALUControl);
    logic RtypeSub;
    assign RtypeSub = funct7b5 & opb5; // TRUE for R-type subtract
instruction

```

```

always_comb
  case(ALUOp)
    2'b00: ALUControl = 4'b0000; // lw, sw, lui (add)
    2'b01: ALUControl = 4'b0001; // beq (sub)
    default: case(func3) // R-type or I-type ALU
      3'b000: if (RtypeSub)
        ALUControl = 4'b0001; // sub
      else
        ALUControl = 4'b0000; // add, addi
      3'b001: ALUControl = 4'b0110; // sll, slli
      3'b101: if (~func7b5)
        ALUControl = 4'b0111; // srl, srli
      else
        ALUControl = 4'b1000; // sra, srai
      3'b010: ALUControl = 4'b0101; // slt, slti
      3'b100: ALUControl = 4'b0100; // xor, xori
      3'b110: ALUControl = 4'b0011; // or, ori
      3'b111: ALUControl = 4'b0010; // and, andi
      default: ALUControl = 4'bxxxx; // ???
    endcase
  endcase
endmodule

module instrdec(input logic [6:0] op, output logic [2:0] immsrc);
  always_comb
    case(op)
      7'b000_0011: immsrc = 3'b000; //lw
      7'b010_0011: immsrc = 3'b001; //sw
      7'b011_0011: immsrc = 3'bxxx; //R-type
      7'b110_0011: immsrc = 3'b010; //beq
      7'b001_0011: immsrc = 3'b000; //I-type
      7'b110_1111: immsrc = 3'b011; //jal
      7'b110_0111: immsrc = 3'b000; //jalr
      7'b011_0111: immsrc = 3'b100; //lui
    endcase
endmodule

/* Data Path Module Definitions */

module regfile(input logic      clk,
               input logic      we3,

```

```

        input logic [ 4:0] a1, a2, a3,
        input logic [31:0] wd3,
        output logic [31:0] rd1, rd2);

    logic [31:0] rf[31:0];

    // three ported register file
    // read two ports combinationaly (A1/RD1, A2/RD2)
    // write third port on rising edge of clock (A3/WD3/WE3)
    // register 0 hardwired to 0

    always_ff @(posedge clk)
        if (we3) rf[a3] <= wd3;

    assign rd1 = (a1 != 0) ? rf[a1] : 0;
    assign rd2 = (a2 != 0) ? rf[a2] : 0;

endmodule

module extend(input logic [31:7] instr,
              input logic [2:0] immsrc,
              output logic [31:0] immext);

    always_comb
        case(immsrc)
            // I-type
            3'b000: immext = {{20{instr[31]}}, instr[31:20]};
            // S-type (stores)
            3'b001: immext = {{20{instr[31]}}, instr[31:25], instr[11:7]};
            // B-type (branches)
            3'b010: immext = {{20{instr[31]}}, instr[7], instr[30:25],
instr[11:8], 1'b0};
            // J-type (jal)
            3'b011: immext = {{12{instr[31]}}, instr[19:12], instr[20],
instr[30:21], 1'b0};
            // U-type (lui)
            3'b100: immext = {{instr[31:12]}, 12'b0};
            default: immext = 32'bx; // undefined
        endcase

endmodule

```

```

module adder(input  [31:0] a, b,
              output [31:0] y);

    assign y = a + b;

endmodule

// no enable parameterized d ff
module flopr #(parameter WIDTH = 8)
    (input logic          clk, reset,
     input logic [WIDTH-1:0] d,
     output logic [WIDTH-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else q <= d;

endmodule

// enable parameterized d ff
module eflopr #(parameter WIDTH = 8)
    (input logic          clk, reset, enable,
     input logic [WIDTH-1:0] d,
     output logic [WIDTH-1:0] q);

    always_ff @(posedge clk, posedge reset)
        if (reset) q <= 0;
        else if (enable) q <= d;

endmodule

module mux2 #(parameter WIDTH = 8)
    (input logic [WIDTH-1:0] d0, d1,
     input logic          s,
     output logic [WIDTH-1:0] y);

    assign y = s ? d1 : d0;

endmodule

```

```

module mux3 #(parameter WIDTH = 8)
    (input  logic [WIDTH-1:0] d0, d1, d2,
     input  logic [1:0]      s,
     output logic [WIDTH-1:0] y);

    assign y = s[1] ? (s[0] ? 0 : d2) : (s[0] ? d1 : d0); // picks 0 with
input 11

```

```

endmodule

```

```

// New idmem which manages a couple control signals.
// Importantly chooses which data for the ReadData signal on figure

```

```

module idmem(input  logic clk, MemWrite, AddrSrc,
             input  logic [31:0] PC, DataAdr, WriteData,
             output logic [31:0] ReadData);

```

```

    logic [31:0] Instr, dmemReadData;

```

```

    imem imem(PC, Instr);

```

```

    dmem dmem(clk, MemWrite, DataAdr, WriteData, dmemReadData);

```

```

    assign ReadData = (AddrSrc != 1) ? Instr : dmemReadData;

```

```

endmodule

```

```

module imem(input  logic [31:0] a,
            output logic [31:0] rd);

```

```

    logic [31:0] RAM[63:0];

```

```

    initial

```

```

        $readmemh("riscv_test_program.txt",RAM);

```

```

    assign rd = RAM[a[31:2]]; // word aligned

```

```

endmodule

```

```

module dmem(input  logic      clk, we,

```

```

        input logic [31:0] a, wd,
        output logic [31:0] rd);

logic [31:0] RAM[63:0];

assign rd = RAM[a[31:2]]; // word aligned

always_ff @(posedge clk)
    if (we) RAM[a[31:2]] <= wd;

endmodule

module alu(input logic [31:0] a, b,
           input logic [3:0] alucontrol,
           output logic [31:0] result,
           output logic zero, lessThan);

logic [31:0] condinvb, sum;
logic v; // overflow
logic isAddSub; // true when is add or subtract operation

assign condinvb = alucontrol[0] ? ~b : b;
assign sum = a + condinvb + alucontrol[0];
assign isAddSub = ~alucontrol[3] & ~alucontrol[2] & ~alucontrol[1] |
~alucontrol[1] & alucontrol[0]; //4'b000X or 4'bXX01

always_comb
    case (alucontrol)
        4'b0000: result = sum; // add
        4'b0001: result = sum; // subtract
        4'b0010: result = a & b; // and
        4'b0011: result = a | b; // or
        4'b0100: result = a ^ b; // xor
        4'b0101: result = sum[31] ^ v; // slt
        4'b0110: result = a << b[4:0]; // sll
        4'b0111: result = a >> b[4:0]; // srl
        4'b1000: result = $unsigned($signed(a) >>> b[4:0]); //sra
        default: result = 32'bx;
    endcase

assign zero = (result == 32'b0);
assign lessThan = sum[31] ^ v;

```

```
    assign v = ~(alucontrol[0] ^ a[31] ^ b[31]) & (a[31] ^ sum[31]) &  
isAddSub;  
  
endmodule
```

riscv_testbench.sv

```

module testbench();

    logic        clk;
    logic        reset;

    logic [31:0] WriteData, DataAdr, PC, Result;
    logic        MemWrite;

    // instantiate device to be tested
    top dut(clk, reset, WriteData, DataAdr, PC, Result, MemWrite);

    // initialize test
    initial
        begin
            reset <= 1; # 22; reset <= 0;
        end

    // generate clock to sequence tests
    always
        begin
            clk <= 1; # 5; clk <= 0; # 5;
        end

    // check results
    always @(negedge clk)
        begin
            if(PC === 32'h68) begin
                $stop;
            end
        end
    end
endmodule

```

riscvextended_display_header.sv

```

module de2_display(output logic [6:0] HEX0, HEX1, HEX2, HEX3, HEX4, HEX5,
HEX6, HEX7, output logic [8:0] LEDG);
    logic [31:0] writedata, dataadr, PC, Result;
    logic memwrite;
    top riscv_multi_cycle(KEY[0], SW[0], writedata, dataadr, PC, Result,
memwrite);
    sevensseg_decoder dec0(Result[3:0], HEX0);
    sevensseg_decoder dec1(Result[7:4], HEX1);
    sevensseg_decoder dec2(Result[11:8], HEX2);
    sevensseg_decoder dec3(Result[15:12], HEX3);
    sevensseg_decoder dec4(Result[19:16], HEX4);
    sevensseg_decoder dec5(Result[23:20], HEX5);
    sevensseg_decoder dec6(Result[27:24], HEX6);
    sevensseg_decoder dec7(Result[31:28], HEX7);
    assign LEDG = PC[8:0];
endmodule

module sevensseg_decoder(input  logic [3:0] data,
                        output logic [6:0] segments);

    always_comb
    case (data)           //Sg - Sa
        4'h0: segments = 7'b1000000;
        4'h1: segments = 7'b1111001;
        4'h2: segments = 7'b0100100;
        4'h3: segments = 7'b0110000;
        4'h4: segments = 7'b0011001;
        4'h5: segments = 7'b0010010;
        4'h6: segments = 7'b0000010;
        4'h7: segments = 7'b1111000;
        4'h8: segments = 7'b0000000;
        4'h9: segments = 7'b0011000;
        4'hA: segments = 7'b0001000;
        4'hB: segments = 7'b0000011;
        4'hC: segments = 7'b0100111;
        4'hD: segments = 7'b0100001;
        4'hE: segments = 7'b0000110;
        4'hF: segments = 7'b0001110;
    endcase
endmodule

```

riscv_test_program.s

```

lui s0, 0x12345
addi s0, s0, 0x678      # s0 = 0x12345678
addi s1, zero, 10       # s1 = 10
addi s2, zero, 5        # s2 = 5
addi s3, zero, 15       # s3 = 15
sub s4, s3, s2          # s4 = 15 - 5 = 10
beq s4, s1, beqlabel    # 10 = 10, should be taken
add s1, zero, zero      # shouldn't happen, s1 = 0
beqlabel:
blt s1, s2, bltlabel    # 10 < 5 ? shouldn't be taken
slli s5, s0, 4          # s5 = 0x23456780
bltlabel:
andi s5, s5, 0x07F      # s5 = 0
jal ra, shiftright
bge s0, s5, bgelabel    # 0x12345678 > 0, should be taken
add s0, zero, zero      # shouldn't happen, s0 = 0
bgelabel:
sw, s0, 0x100(zero)     # mem[100] = 0x12345678
lw, s6, 0x100(zero)     # s6 = mem[100] = 0x12345678
slli s6, s6, 3          # s6 = 0x91A2B3C0
srai, s6, s6, 4         # s6 = 0xF91A2B3C
slt s7, s2, s1          # s7 = 5 < 10 = 1
slt s7, s1, s2          # s7 = 10 < 5 = 0
xori s8, s3, 7          # s8 = 1111 ^ 0111 = 0x8
or s9, s1, s2           # s9 = 1010 | 0101 = 0xF
bne zero, s0, end
shiftright:             # function. current state should be: s0 = 0x12345678, s3 =
15
srl s5, s0, s3           # s5 = 0x00002468
jalr zero, ra, 0
end:
beq zero, zero, end

```

riscv_test_program.txt

```
12345437
67840413
00A00493
00500913
00F00993
41298A33
009A0463
000004B3
0124C463
00441A93
07FAFA93
030000EF
01545463
00000433
06802223
06402B03
003B1B13
404B5B13
00992BB3
0124ABB3
0079CC13
0124ECB3
00801663
01345AB3
00008067
00000063
```